

MODULE 5 · LEVEL 300



Testing: Proving It Works with AI

From working code to a suite you can trust - unit, integration, end-to-end, data and resilience, with GitHub Copilot

Day 2 · 90 minutes · Builds on Module 4 (the code) & Module 2 (the criteria) · Novnex



Where we are, and today's shape

⌚ approx 2 min

Module 4 built the feature. Today we prove it works - and turn acceptance criteria into a suite you can trust.



Framing

Why AI changes testing, and the map across the pyramid.

~16 min



Testing with Copilot

Unit, API, e2e, data, prediction, resilience, maintenance.

~52 min



Apply

By track, anti-patterns, the hand-off, and the lab.

~22 min

The thread from Module 2 pays off here: good acceptance criteria are exactly what good tests are built from.



Learning objectives

 approx 2 min

By the end of this module you will be able to:



Test from the criteria

Turn Given/When/Then acceptance criteria into real, meaningful tests.



Cover the pyramid

Use AI across unit, integration/API and end-to-end - at the right level.



Make tests resilient

Generate data, reduce flakiness, and keep the suite maintainable.



Keep green meaningful

Make tests verify intent - so a passing suite actually means quality.



Why AI changes testing

 approx 4 min



Coverage got cheap

- **The excuse is gone:**
- 'no time to write tests' - Copilot drafts them in seconds.
- **From code or spec:**
- generate tests from a function, or from the criteria.
- **Every level:**
- unit, integration, end-to-end, even test data.
- **Maintenance too:**
- resilient, self-healing tests cut the upkeep tax.



But green must mean good

- AI can write tests that just mirror the code's bugs
- High coverage is not the same as high quality
- A test that never fails is proving nothing
- **So tests must verify intent - not just execution**



Where AI helps across testing

 approx 4 min



Unit tests

From code or from acceptance criteria.



Integration & API

Tests to the contract you designed.



End-to-end / UI

Playwright and Selenium flows.



Test data

Realistic, edge-case, privacy-safe.



Defect prediction

Focus effort where bugs cluster.



Resilience

Self-healing scripts, less flakiness.

We'll take these in order. The through-line: every test should trace back to a requirement.



The testing pyramid - and where AI plugs in

⌚ approx 4 min



Unit

Many, fast, cheap

AI: Generate from code & criteria; cover edge cases



Integration / API

Fewer; test the seams

AI: Generate from the OpenAPI contract (Module 3)



End-to-end

Few; test the journey

AI: Generate & self-heal UI flows; keep them stable

PART 2

Testing with Copilot

Unit, API, end-to-end, data, prediction and resilience - each with a real example.

approx 52 min





Unit tests - from code and from criteria

⌚ approx 5 min

Two ways in: /tests over a function, or - better - generate tests from the acceptance criteria you wrote in Module 2.

From criteria to tests

```
# Given whole == 0, percentage() raises
def test_percentage_zero_whole():
    with pytest.raises(ValueError):
        percentage(1, 0)
# also: empty list, negative, boundary
```

Make them meaningful

- **From the criteria:**
 - each Given/When/Then becomes a test.
- **Push for edges:**
 - ask for boundary and negative cases.
- **Not just the happy path:**
 - that's where AI stops by default.
- **Read them:**
 - a test you didn't check proves nothing.



Test design from acceptance criteria

🕒 approx 5 min

One criterion -> a set of cases

Criterion: reset link expires after 30 min

- valid link at 29 min -> accepted
- link at 31 min -> rejected
- already-used link -> rejected
- tampered token -> rejected
- unknown email -> generic reply

Coverage that means something

- **Traceable:**
 - each test maps to a criterion from Module 2.
- **Boundaries:**
 - AI is great at enumerating the edges.
- **Ask 'what did we miss?':**
 - a strong prompt for gaps.
- **You judge relevance:**
 - keep the cases that matter.



Integration & API tests from the contract

🕒 approx 4 min

Test to the OpenAPI contract

```
POST /password-reset {email: known}
  -> 202 Accepted
POST /password-reset {email: unknown}
  -> 202 (generic, no disclosure)
Nth request -> 429 (threshold TBC)
```

The contract is the test plan

- **From Module 3:**
 - the OpenAPI spec defines every response.
- **Test the seams:**
 - where components meet is where bugs hide.
- **Contract tests:**
 - catch breaking changes early.
- **Realistic, not exhaustive:**
 - cover the meaningful paths.



End-to-end and UI tests

⌚ approx 4 min

Playwright (generated)

```
await page.goto('/reset')
await page.fill('#email', user.email)
await page.click('text=Send reset link')
await expect(page.getByText(
  'Check your email')).toBeVisible()
```

Test the journey

- **From a flow description:**
 - Copilot drafts the Playwright script.
- **Fewer, higher-value:**
 - e2e is slow - test key journeys only.
- **Accessibility & mobile:**
 - generate those checks too.
- **Stability matters:**
 - which the next slide is all about.



Test data - realistic, edge, and safe

⌚ approx 4 min

Generate the awkward cases

```
"Give me 20 customer records:  
valid, unicode names, empty email,  
legacy (no email), very long strings.  
Synthetic only - no real PII."
```

Data is half the test

- **Edge cases on demand:**
 - the weird inputs you'd never think of.
- **Realistic shapes:**
 - data that behaves like production.
- **Privacy-safe:**
 - synthetic data - never copy real PII.
- **Fixtures & factories:**
 - generate reusable builders.



Resilient and self-healing tests

 approx 4 min



Cut the flakiness tax

- **Better locators:**
 - AI suggests robust selectors over brittle ones.
- **Self-healing:**
 - tests adapt when a locator changes slightly.
- **Diagnose flakes:**
 - 'why is this test intermittent?'
- **Auto-repair drafts:**
 - propose a fix when the UI shifts.



Keep it honest

- Self-healing must not hide a real regression
- A test that always 'heals' green tests nothing
- Review healing changes like any other diff
- **Stability is the goal - not silence**



Defect prediction & risk-based testing

 approx 4 min



Find the hotspots

AI flags files that change often or are highly complex.



Spend effort well

Test the risky, high-blast-radius areas harder.



Prioritise cases

Rank scenarios by likelihood and impact.



Close coverage gaps

'Which criteria have no test?' - then fill them.



Visual testing with AI

 approx 3 min



Catch what asserts miss

- **Perceptual diffs:**
 - AI compares screenshots and flags real changes.
- **Ignore the noise:**
 - smarter than pixel-diff about trivial shifts.
- **Responsive & a11y:**
 - check layouts across sizes and contrast.
- **Great for UI-heavy work:**
 - the portal, dashboards, reports.



Where it fits

- Complements functional tests - it doesn't replace them
- Best on stable, high-traffic screens
- A human approves the intended visual changes
- [Cuts manual 'does it still look right?' checking](#)



Maintaining the suite

⌚ approx 4 min



Tame flaky tests

Find, explain and stabilise intermittent failures.



Refactor tests

Keep tests readable as the code evolves.



Fill coverage gaps

Spot untested paths and generate the missing cases.



Prune dead weight

Remove redundant tests that only slow the suite.

A suite is a living asset - AI makes the upkeep affordable, so tests stay trusted instead of ignored.



Live demo: criteria to a trusted test

⌚ approx 6 min

Watch coverage become confidence

- Take an acceptance criterion from Module 2
- Generate unit tests - including the edges
- Run them; one fails and finds a real bug
- Fix the code; the test goes green
- Ask 'which criteria still have no test?'

Flow

- **1.**
 - Pick a criterion.
- **2.**
 - Generate tests + edges.
- **3.**
 - Run - watch one fail.
- **4.**
 - Fix the code, go green.
- **5.**
 - Check for coverage gaps.



Responsible testing with AI

⌚ approx 5 min



Tests that mirror bugs

AI writes a test asserting the buggy behaviour.

Guard: Test against the criteria and intent, not the current code.



Coverage theatre

Chasing a percentage, not real quality.

Guard: Value meaningful assertions over line coverage.



Blind trust in green

Assuming a passing suite means it works.

Guard: Review generated tests; make sure they can actually fail.



AI grading AI

The same model writes and 'approves' the code.

Guard: Keep a human in the loop; tests are an independent check.

PART 3

Apply it

By track, the anti-patterns, the hand-off to deployment, and your lab.

approx 22 min





Testing with Copilot - by track

 approx 4 min



Web / Drupal

Domain/unit now; API/E2E when a running portal exists.



Data & Analytics

Defined dirty-row tests plus known-good metrics; pipeline gaps named.



Power BI

Validate Python reference calculations; DAX/model checks remain gaps.



RPA

Regression tests for gates, call order and CRM failure; recovery is a gap.



Anti-patterns to avoid

 approx 4 min



Don't

- Let AI test the code by copying it
- Chase a coverage number for its own sake
- Trust green without reading the tests
- Let self-healing hide real regressions
- Generate a thousand shallow, duplicate tests



Do

- Write tests against the criteria and intent
- Insist on edge and negative cases
- Make sure each test can actually fail
- Keep the suite fast, stable and trusted
- Trace tests back to requirements



The hand-off to deployment

 approx 3 min

A trustworthy suite is the gate into release. If the tests are meaningful, CI/CD can move fast and safely.



Criteria covered

Every acceptance criterion has a meaningful test.



Balanced suite

Unit, integration and a few e2e - at the right levels.



Stable & fast

Low flakiness, quick enough to run on every change.



Gate-ready

Green is a real signal your pipeline can rely on.

Module 6 puts this suite into a pipeline - failing tests should stop a release, not slow one down.



Your lab: build a suite you trust

 approx 2 min

Lab 5 (this afternoon) - on your own track

- Take an acceptance criterion (from Lab 2) and generate tests - including edges
- Run them; find and fix at least one real gap or bug
- Add a runnable track test; record unavailable layers as gaps
- Ask which supported criterion has no test - and close one

Role groups; pair up. The goal is a small suite whose green you'd actually trust.



Discussion & reflection

 approx 5 min

Talk to the person next to you - three minutes:

- Where does your team have the weakest test coverage - and how would AI help you close it?
- What would make you trust a passing suite - and what makes you distrust one today?

Coverage gaps

Flaky tests

Trust in green

Test data

Maintenance

We'll take a few answers before wrapping up.



Key takeaways

- AI makes coverage cheap - so 'no time for tests' is over.
- Test from the acceptance criteria, across the whole pyramid.
- Generate the data and the edges; keep tests resilient and fast.
- Focus effort with defect prediction; maintain the suite so it stays trusted.
- Green must mean good - tests verify intent, and a human stays in the loop.

What's next

Module 6

Deployment

then

Lab 5 this afternoon

Questions?